Source Code: scanner.py

Project: Automated Web Scanner (Secure Headers & CORS Flaws Audit)
Author: Tanisha | GitHub: tsh471/automated-web-scanner
Event: IBM SkillsBuild National Cybersecurity Bootcamp

```
#!/usr/bin/env python3
"""
Automated Web Security Scanner: Secure Headers and CORS Flaws Auditor
=====================================================================
Project: IBM SkillsBuild National Cybersecurity Bootcamp
Author: Cybersecurity Bootcamp Participant
Topic: 2. Automated Web Scanner: Programmed a Python script to audit sites for secure headers and CORS
flaws.

Description:
    A high-performance, zero-external-dependency automated security scanner designed
    to audit web applications for modern HTTP security headers, information leakage,
    and Cross-Origin Resource Sharing (CORS) misconfigurations according to OWASP standards.

Features:
    - Comprehensive Secure Headers Inspection (CSP, HSTS, XFO, XCTO, Referrer-Policy, Permissions-Polic
y, COOP/COEP/CORP)
    - Sensitive Information Leakage Detection (Server banners, X-Powered-By, technology fingerprints)
    - Active CORS Vulnerability Probing (Arbitrary Origin Reflection, Null Origin Trust, Credential Lea
kage, Suffix/Prefix Bypasses)
    - Security Scoring Algorithm (0-100 score with A+ through F grading)
    - Multi-target concurrent scanning via ThreadPoolExecutor
    - Output formats: Rich ANSI Color Terminal, Structured JSON, and Standalone HTML Reports
    - Built-in --demo mode for offline evaluation and grading verification
"""

import argparse
import concurrent.futures
from dataclasses import dataclass, field, asdict
import datetime
import html
import http.client
import json
import os
import re
import ssl
import sys
import time
import urllib.error
import urllib.parse
import urllib.request
from typing import List, Dict, Optional, Tuple, Any

# ============================================================================
# ANSI Color Formatting & Terminal Helpers
# ============================================================================

class Colors:
    RESET = "\033[0m"
    BOLD = "\033[1m"
    DIM = "\033[2m"
    UNDERLINE = "\033[4m"

    RED = "\033[91m"
    GREEN = "\033[92m"
    YELLOW = "\033[93m"
    BLUE = "\033[94m"
    MAGENTA = "\033[95m"
    CYAN = "\033[96m"
```

```
    WHITE = "\033[97m"

def colorize(text: str, color_code: str, enable_color: bool = True) -> str:
    if not enable_color or not sys.stdout.isatty():
        return text
    return f"{color_code}{text}{Colors.RESET}"

# ==============================================================================
# Data Structures
# ==============================================================================

class Severity:
    CRITICAL = "CRITICAL"
    HIGH = "HIGH"
    MEDIUM = "MEDIUM"
    LOW = "LOW"
    INFO = "INFO"
    PASS = "PASS"

@dataclass
class HeaderFinding:
    header: str
    status: str           # PASS, WARN, FAIL, INFO
    severity: str         # CRITICAL, HIGH, MEDIUM, LOW, INFO
    value: Optional[str]
    description: str
    remediation: str

@dataclass
class CORSFinding:
    probe_type: str       # e.g., "Reflected Origin", "Null Origin Trust", "Credentials Leakage"
    test_origin: str
    severity: str
    vulnerable: bool
    description: str
    acao_header: Optional[str]
    acac_header: Optional[str]
    remediation: str

@dataclass
class ScanResult:
    target_url: str
    scan_timestamp: str
    status_code: int
    response_time_ms: float
    server_ip: Optional[str] = None
    security_score: int = 100
    grade: str = "A+"
    header_findings: List[HeaderFinding] = field(default_factory=list)
    cors_findings: List[CORSFinding] = field(default_factory=list)
    raw_headers: Dict[str, str] = field(default_factory=dict)
    errors: List[str] = field(default_factory=list)

# ==============================================================================
# Security Headers Audit Engine
# ==============================================================================

class HeaderAuditor:
    """Evaluates HTTP response headers against OWASP security guidelines."""

    @staticmethod
    def audit(headers: Dict[str, str], url: str) -> Tuple[List[HeaderFinding], int]:
        findings: List[HeaderFinding] = []
        score_deductions = 0
        norm_headers = {k.lower(): v for k, v in headers.items()}
        is_https = url.lower().startswith("https://")
```

```
        # 1. Content-Security-Policy (CSP)
        csp = norm_headers.get("content-security-policy")
        if not csp:
            findings.append(HeaderFinding(
                header="Content-Security-Policy",
                status="FAIL",
                severity=Severity.HIGH,
                value=None,
                description="Missing CSP header. The application is vulnerable to Cross-Site Scripting
(XSS), Clickjacking, and unauthorized data injection.",
                remediation="Implement a robust CSP such as: default-src 'self'; script-src 'self'; obj
ect-src 'none'; base-uri 'self'; frame-ancestors 'none';"
            ))
            score_deductions += 25
        else:
            issues = []
            if "'unsafe-inline'" in csp:
                issues.append("allows 'unsafe-inline' scripts")
            if "'unsafe-eval'" in csp:
                issues.append("allows 'unsafe-eval' dynamic execution")
            if "default-src" not in csp and "script-src" not in csp:
                issues.append("missing default-src and script-src fallback directives")
            if "object-src 'none'" not in csp:
                issues.append("missing object-src 'none' (allows Flash/Java plugin embedding)")

            if issues:
                findings.append(HeaderFinding(
                    header="Content-Security-Policy",
                    status="WARN",
                    severity=Severity.MEDIUM,
                    value=csp[:80] + ("..." if len(csp) > 80 else ""),
                    description=f"CSP is present but contains potential weaknesses: {', '.join(issue
s)}.",
                    remediation="Eliminate 'unsafe-inline' using nonces or hashes, disable 'unsafe-eva
l', and set object-src 'none'."
                ))
                score_deductions += 10
            else:
                findings.append(HeaderFinding(
                    header="Content-Security-Policy",
                    status="PASS",
                    severity=Severity.PASS,
                    value=csp[:80] + ("..." if len(csp) > 80 else ""),
                    description="Strong Content-Security-Policy header is configured.",
                    remediation="Maintain existing restrictive CSP directives."
                ))

        # 2. Strict-Transport-Security (HSTS)
        hsts = norm_headers.get("strict-transport-security")
        if not hsts:
            if is_https:
                findings.append(HeaderFinding(
                    header="Strict-Transport-Security",
                    status="FAIL",
                    severity=Severity.HIGH,
                    value=None,
                    description="Missing HSTS header on HTTPS connection. Leaves users vulnerable to SS
L-stripping and Man-in-the-Middle (MITM) downgrade attacks.",
                    remediation="Add: Strict-Transport-Security: max-age=31536000; includeSubDomains; p
reload"
                ))
                score_deductions += 20
            else:
                findings.append(HeaderFinding(
                    header="Strict-Transport-Security",
                    status="INFO",
                    severity=Severity.INFO,
```

```
                    value=None,
                    description="Target is served over plain HTTP. HSTS headers are only valid over sec
ure HTTPS connections.",
                    remediation="Migrate entire site to HTTPS and configure HSTS."
                ))
        else:
            max_age_match = re.search(r"max-age=(\d+)", hsts, re.IGNORECASE)
            max_age = int(max_age_match.group(1)) if max_age_match else 0
            if max_age < 15552000:  # < 180 days
                findings.append(HeaderFinding(
                    header="Strict-Transport-Security",
                    status="WARN",
                    severity=Severity.LOW,
                    value=hsts,
                    description=f"HSTS max-age is set to {max_age} seconds, which is less than recommen
ded minimum (180 days / 15552000s).",
                    remediation="Increase HSTS max-age to at least 31536000 (1 year) with includeSubDom
ains."
                ))
                score_deductions += 5
            else:
                findings.append(HeaderFinding(
                    header="Strict-Transport-Security",
                    status="PASS",
                    severity=Severity.PASS,
                    value=hsts,
                    description="Strong HSTS configuration enforces encrypted connections.",
                    remediation="Keep HSTS enabled and consider submitting domain to Chrome preload lis
t."
                ))

        # 3. X-Frame-Options (XFO)
        xfo = norm_headers.get("x-frame-options")
        if not xfo:
            if csp and "frame-ancestors" in csp:
                findings.append(HeaderFinding(
                    header="X-Frame-Options",
                    status="PASS",
                    severity=Severity.PASS,
                    value="Handled via CSP frame-ancestors",
                    description="Legacy X-Frame-Options is absent, but modern CSP 'frame-ancestors' dir
ective is active.",
                    remediation="Optional: Add 'X-Frame-Options: DENY' for legacy browser fallback."
                ))
            else:
                findings.append(HeaderFinding(
                    header="X-Frame-Options",
                    status="FAIL",
                    severity=Severity.MEDIUM,
                    value=None,
                    description="Missing X-Frame-Options and CSP frame-ancestors. The page can be embed
ded in an iframe, enabling Clickjacking attacks.",
                    remediation="Set: X-Frame-Options: DENY or SAMEORIGIN (or use CSP frame-ancestors
'none')."
                ))
                score_deductions += 15
        else:
            if xfo.upper() in ["DENY", "SAMEORIGIN"]:
                findings.append(HeaderFinding(
                    header="X-Frame-Options",
                    status="PASS",
                    severity=Severity.PASS,
                    value=xfo,
                    description=f"Properly configured with '{xfo}' to prevent UI redressing / clickjack
ing.",
                    remediation="Keep existing policy in place."
                ))
```

```
            else:
                findings.append(HeaderFinding(
                    header="X-Frame-Options",
                    status="WARN",
                    severity=Severity.LOW,
                    value=xfo,
                    description=f"Unusual or obsolete X-Frame-Options value: '{xfo}'.",
                    remediation="Replace with 'DENY' or 'SAMEORIGIN'."
                ))
                score_deductions += 5

        # 4. X-Content-Type-Options
        xcto = norm_headers.get("x-content-type-options")
        if not xcto:
            findings.append(HeaderFinding(
                header="X-Content-Type-Options",
                status="FAIL",
                severity=Severity.MEDIUM,
                value=None,
                description="Missing X-Content-Type-Options header. Browsers may perform MIME-type snif
fing, transforming non-executable files into executable scripts.",
                remediation="Set: X-Content-Type-Options: nosniff"
            ))
            score_deductions += 10
        elif "nosniff" in xcto.lower():
            findings.append(HeaderFinding(
                header="X-Content-Type-Options",
                status="PASS",
                severity=Severity.PASS,
                value=xcto,
                description="MIME-sniffing protection is properly enabled ('nosniff').",
                remediation="Maintain existing header configuration."
            ))
        else:
            findings.append(HeaderFinding(
                header="X-Content-Type-Options",
                status="WARN",
                severity=Severity.LOW,
                value=xcto,
                description=f"X-Content-Type-Options contains unexpected value '{xcto}'. Expected 'nosn
iff'.",
                remediation="Ensure value is strictly set to 'nosniff'."
            ))
            score_deductions += 5

        # 5. Referrer-Policy
        ref_pol = norm_headers.get("referrer-policy")
        if not ref_pol:
            findings.append(HeaderFinding(
                header="Referrer-Policy",
                status="WARN",
                severity=Severity.LOW,
                value=None,
                description="Missing Referrer-Policy header. Sensitive URL paths, tokens, or query para
meters may leak in HTTP Referer headers to external origins.",
                remediation="Set: Referrer-Policy: strict-origin-when-cross-origin (or no-referrer)."
            ))
            score_deductions += 5
        elif ref_pol.lower() in ["unsafe-url", "no-referrer-when-downgrade"]:
            findings.append(HeaderFinding(
                header="Referrer-Policy",
                status="FAIL",
                severity=Severity.MEDIUM,
                value=ref_pol,
                description=f"Weak Referrer-Policy '{ref_pol}' leaks full URL paths and parameters acro
ss origins.",
                remediation="Change to: Referrer-Policy: strict-origin-when-cross-origin"
```

```
            ))
            score_deductions += 8
        else:
            findings.append(HeaderFinding(
                header="Referrer-Policy",
                status="PASS",
                severity=Severity.PASS,
                value=ref_pol,
                description=f"Secure Referrer-Policy '{ref_pol}' restricts referral data leakage.",
                remediation="Maintain safe Referrer-Policy."
            ))

        # 6. Permissions-Policy (Feature-Policy)
        perm_pol = norm_headers.get("permissions-policy") or norm_headers.get("feature-policy")
        if not perm_pol:
            findings.append(HeaderFinding(
                header="Permissions-Policy",
                status="WARN",
                severity=Severity.LOW,
                value=None,
                description="Missing Permissions-Policy. Browser features (camera, microphone, geolocat
ion) are not explicitly restricted.",
                remediation="Define policy: Permissions-Policy: camera=(), microphone=(), geolocation=
()"
            ))
            score_deductions += 5
        else:
            findings.append(HeaderFinding(
                header="Permissions-Policy",
                status="PASS",
                severity=Severity.PASS,
                value=perm_pol[:60] + ("..." if len(perm_pol) > 60 else ""),
                description="Permissions-Policy header is configured to control browser API access.",
                remediation="Periodically review permitted features."
            ))

        # 7. Cross-Origin Headers (COOP / COEP / CORP)
        coop = norm_headers.get("cross-origin-opener-policy")
        if not coop:
            findings.append(HeaderFinding(
                header="Cross-Origin-Opener-Policy (COOP)",
                status="INFO",
                severity=Severity.INFO,
                value=None,
                description="Missing COOP header. The site cannot guarantee process isolation from cros
s-origin window references (Spectre mitigation).",
                remediation="Consider setting: Cross-Origin-Opener-Policy: same-origin"
            ))
        else:
            findings.append(HeaderFinding(
                header="Cross-Origin-Opener-Policy",
                status="PASS",
                severity=Severity.PASS,
                value=coop,
                description=f"Cross-Origin isolation enforced via COOP: {coop}.",
                remediation="Keep COOP enabled."
            ))

        # 8. Information Disclosure Headers (Server, X-Powered-By, etc.)
        leaky_headers = ["server", "x-powered-by", "x-aspnet-version", "x-generator", "x-runtime"]
        detected_leaks = [f"{h}: {norm_headers[h]}" for h in leaky_headers if h in norm_headers]
        if detected_leaks:
            findings.append(HeaderFinding(
                header="Information Leakage",
                status="FAIL",
                severity=Severity.LOW,
                value="; ".join(detected_leaks),
```

```
                description=f"Server exposes internal software versions in response headers: {', '.join
(detected_leaks)}. Aids attackers in targeted exploitation.",
                remediation="Disable server tokens and remove 'X-Powered-By' headers in web server/fram
ework configurations."
            ))
            score_deductions += 5
        else:
            findings.append(HeaderFinding(
                header="Information Leakage",
                status="PASS",
                severity=Severity.PASS,
                value="None detected",
                description="No sensitive server banners or software version headers detected.",
                remediation="Ensure production servers suppress verbose banners."
            ))

        # 9. Deprecated Headers (X-XSS-Protection)
        xxss = norm_headers.get("x-xss-protection")
        if xxss and "1; mode=block" in xxss:
            findings.append(HeaderFinding(
                header="X-XSS-Protection",
                status="INFO",
                severity=Severity.INFO,
                value=xxss,
                description="Legacy X-XSS-Protection is enabled. Modern browsers rely on CSP instead of
this deprecated header.",
                remediation="Ensure Content-Security-Policy is prioritized."
            ))

        final_score = max(0, 100 - score_deductions)
        return findings, final_score

# ============================================================================
# CORS Vulnerability Audit Engine
# ============================================================================

class CORSAuditor:
    """Active prober for Cross-Origin Resource Sharing (CORS) misconfigurations."""

    @staticmethod
    def audit(target_url: str, timeout: float = 8.0, insecure_ssl: bool = False) -> List[CORSFinding]:
        findings: List[CORSFinding] = []
        parsed = urllib.parse.urlparse(target_url)
        target_domain = parsed.hostname or "example.com"
        target_scheme = parsed.scheme or "https"

        ssl_ctx = ssl.create_default_context()
        if insecure_ssl:
            ssl_ctx.check_hostname = False
            ssl_ctx.verify_mode = ssl.CERT_NONE

        # Define probe vectors
        probes = [
            {
                "type": "Arbitrary Origin Reflection",
                "origin": "https://attacker-security-test.com",
                "desc": "Testing if server blindly reflects untrusted third-party origin."
            },
            {
                "type": "Null Origin Trust",
                "origin": "null",
                "desc": "Testing if server trusts 'null' origin (vulnerable to sandboxed iframes & dat
a: URI exploits)."
            },
            {
                "type": "Domain Suffix Regex Bypass",
                "origin": f"https://{target_domain}.attacker-test.com",
```

```
                "desc": f"Testing flawed regex validation where origin ends with target domain name."
            },
            {
                "type": "Domain Prefix Regex Bypass",
                "origin": f"https://attacker-{target_domain}",
                "desc": f"Testing flawed regex where origin begins with or contains target domain nam
e."
            },
        ]

        if target_scheme == "https":
            probes.append({
                "type": "Insecure HTTP Protocol Trust",
                "origin": f"http://{target_domain}",
                "desc": "Testing if HTTPS endpoint allows cleartext HTTP origin (MITM vulnerability)."
            })

        for p in probes:
            finding = CORSAuditor._probe_single_origin(
                target_url=target_url,
                probe_type=p["type"],
                test_origin=p["origin"],
                description=p["desc"],
                ssl_ctx=ssl_ctx,
                timeout=timeout
            )
            if finding:
                findings.append(finding)

        # If no vulnerabilities found across all probes
        if not any(f.vulnerable for f in findings):
            findings.append(CORSFinding(
                probe_type="CORS Security Baseline",
                test_origin="N/A",
                severity=Severity.PASS,
                vulnerable=False,
                description="Endpoint does not insecurely reflect arbitrary origins or trust 'null' ori
gin.",
                acao_header=None,
                acac_header=None,
                remediation="Maintain strict origin whitelisting on all sensitive API endpoints."
            ))

        return findings

    @staticmethod
    def _probe_single_origin(target_url: str, probe_type: str, test_origin: str,
                             description: str, ssl_ctx: ssl.SSLContext, timeout: float) -> Optional[COR
SFinding]:
        try:
            req = urllib.request.Request(
                target_url,
                headers={
                    "Origin": test_origin,
                    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) CybersecurityScanner/1.0",
                    "Accept": "*/*"
                },
                method="GET"
            )
            with urllib.request.urlopen(req, timeout=timeout, context=ssl_ctx) as response:
                resp_headers = {k.lower(): v for k, v in response.headers.items()}
                acao = resp_headers.get("access-control-allow-origin")
                acac = resp_headers.get("access-control-allow-credentials", "").lower()

                if not acao:
                    # Endpoint did not return CORS headers for this origin -> Safe
                    return None
```

```
                # Analysis 1: Arbitrary Origin Reflected WITH Credentials
                if (acao.strip() == test_origin or acao.strip() == "*") and acac == "true":
                    if acao.strip() == "*":
                        return CORSFinding(
                            probe_type=f"{probe_type} + Credentials",
                            test_origin=test_origin,
                            severity=Severity.HIGH,
                            vulnerable=True,
                            description=f"Server returned wildcard ACAO with credentials allowed (ACAC:
true). Flawed authorization logic.",
                            acao_header=acao,
                            acac_header=acac,
                            remediation="Do not combine wildcard ACAO with Access-Control-Allow-Credent
ials: true."
                        )
                    else:
                        return CORSFinding(
                            probe_type=f"{probe_type} with Credentials (CRITICAL)",
                            test_origin=test_origin,
                            severity=Severity.CRITICAL,
                            vulnerable=True,
                            description=f"Server reflects arbitrary Origin '{test_origin}' AND enables
credentials (ACAC: true). Cross-origin attackers can steal private authenticated user sessions.",
                            acao_header=acao,
                            acac_header=acac,
                            remediation="Implement strict server-side whitelist matching against truste
d origins. Never dynamically reflect untrusted Origin headers with credentials."
                        )

                # Analysis 2: Arbitrary Origin Reflected WITHOUT Credentials
                if acao.strip() == test_origin:
                    sev = Severity.HIGH if probe_type == "Null Origin Trust" else Severity.MEDIUM
                    return CORSFinding(
                        probe_type=probe_type,
                        test_origin=test_origin,
                        severity=sev,
                        vulnerable=True,
                        description=f"Server dynamically reflected untrusted origin '{test_origin}' in
Access-Control-Allow-Origin.",
                        acao_header=acao,
                        acac_header=acac or "false",
                        remediation="Do not dynamically mirror the Request Origin. Validate against an
explicit whitelist of trusted domains."
                    )

                # Analysis 3: Null Origin explicitly trusted
                if test_origin == "null" and acao.strip() == "null":
                    return CORSFinding(
                        probe_type="Null Origin Trust",
                        test_origin="null",
                        severity=Severity.HIGH,
                        vulnerable=True,
                        description="Server explicitly trusts 'null' origin. Allows sandbox-escaped ifr
ames, local files, and data: URIs to access protected resources.",
                        acao_header=acao,
                        acac_header=acac or "false",
                        remediation="Never whitelist or return 'Access-Control-Allow-Origin: null'."
                    )

                # Analysis 4: Wildcard origin without credentials
                if acao.strip() == "*":
                    return CORSFinding(
                        probe_type=f"{probe_type} (Wildcard)",
                        test_origin=test_origin,
                        severity=Severity.LOW,
                        vulnerable=False,
```

```
                        description="Server uses wildcard ('*') CORS policy. Acceptable for public read
-only content, but insecure if endpoint serves private user data.",
                        acao_header=acao,
                        acac_header=acac or "false",
                        remediation="If this endpoint returns confidential or user-specific data, repla
ce wildcard with authenticated origin checks."
                    )

        except urllib.error.HTTPError as e:
            headers = {k.lower(): v for k, v in e.headers.items()}
            acao = headers.get("access-control-allow-origin")
            acac = headers.get("access-control-allow-credentials", "").lower()
            if acao == test_origin and acac == "true":
                return CORSFinding(
                    probe_type=f"{probe_type} (Error Response)",
                    test_origin=test_origin,
                    severity=Severity.HIGH,
                    vulnerable=True,
                    description=f"Server reflected origin with credentials on HTTP error status {e.cod
e}.",
                    acao_header=acao,
                    acac_header=acac,
                    remediation="Ensure error handlers do not inadvertently echo Origin headers with cr
edentials."
                )
        except Exception:
            pass

        return None

# ==============================================================================
# Security Scanner Orchestrator
# ==============================================================================

class WebScanner:
    """Manages scan execution against single or multiple targets."""

    def __init__(self, timeout: float = 10.0, insecure_ssl: bool = False, max_threads: int = 5):
        self.timeout = timeout
        self.insecure_ssl = insecure_ssl
        self.max_threads = max_threads

    def scan_url(self, target_url: str) -> ScanResult:
        if not target_url.startswith(("http://", "https://")):
            target_url = "https://" + target_url

        timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        result = ScanResult(
            target_url=target_url,
            scan_timestamp=timestamp,
            status_code=0,
            response_time_ms=0.0
        )

        ssl_ctx = ssl.create_default_context()
        if self.insecure_ssl:
            ssl_ctx.check_hostname = False
            ssl_ctx.verify_mode = ssl.CERT_NONE

        req = urllib.request.Request(
            target_url,
            headers={"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) CybersecurityScanner/1.
0"},
            method="GET"
        )

        start_time = time.time()
```

```
            try:
                with urllib.request.urlopen(req, timeout=self.timeout, context=ssl_ctx) as response:
                    elapsed = (time.time() - start_time) * 1000
                    result.status_code = response.getcode()
                    result.response_time_ms = round(elapsed, 2)
                    result.raw_headers = dict(response.headers)
            except urllib.error.HTTPError as e:
                elapsed = (time.time() - start_time) * 1000
                result.status_code = e.code
                result.response_time_ms = round(elapsed, 2)
                result.raw_headers = dict(e.headers)
                result.errors.append(f"HTTP Error {e.code}: {e.reason}")
            except Exception as e:
                result.errors.append(f"Connection Failed: {str(e)}")
                result.security_score = 0
                result.grade = "F"
                return result

            # Run Header Audit
            header_findings, score = HeaderAuditor.audit(result.raw_headers, target_url)
            result.header_findings = header_findings

            # Run CORS Audit
            cors_findings = CORSAuditor.audit(target_url, timeout=self.timeout, insecure_ssl=self.insecure_
    ssl)
            result.cors_findings = cors_findings

            # Deduct score for CORS vulnerabilities
            for c in cors_findings:
                if c.vulnerable:
                    if c.severity == Severity.CRITICAL:
                        score = max(0, score - 30)
                    elif c.severity == Severity.HIGH:
                        score = max(0, score - 20)
                    elif c.severity == Severity.MEDIUM:
                        score = max(0, score - 10)

            result.security_score = max(0, min(100, score))
            result.grade = self._calculate_grade(result.security_score)

            return result

        def scan_bulk(self, urls: List[str]) -> List[ScanResult]:
            results: List[ScanResult] = []
            with concurrent.futures.ThreadPoolExecutor(max_workers=self.max_threads) as executor:
                future_to_url = {executor.submit(self.scan_url, url): url for url in urls}
                for future in concurrent.futures.as_completed(future_to_url):
                    url = future_to_url[future]
                    try:
                        res = future.result()
                        results.append(res)
                    except Exception as e:
                        failed = ScanResult(
                            target_url=url,
                            scan_timestamp=datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                            status_code=0,
                            response_time_ms=0.0,
                            security_score=0,
                            grade="F",
                            errors=[str(e)]
                        )
                        results.append(failed)
            return results

        @staticmethod
        def _calculate_grade(score: int) -> str:
            if score >= 95:
```

```
            return "A+"
        if score >= 85:
            return "A"
        if score >= 75:
            return "B"
        if score >= 60:
            return "C"
        if score >= 45:
            return "D"
        return "F"

# ==============================================================================
# Built-in Mock / Demo Mode Generator
# ==============================================================================

def generate_demo_results() -> List[ScanResult]:
    """Generates realistic demonstration scan results for offline evaluation."""
    res1 = ScanResult(
        target_url="https://vulnerable-portal.local/api",
        scan_timestamp=datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        status_code=200,
        response_time_ms=142.5,
        security_score=35,
        grade="F",
        header_findings=[
            HeaderFinding(
                header="Content-Security-Policy",
                status="FAIL",
                severity=Severity.HIGH,
                value=None,
                description="Missing CSP header. The application is vulnerable to Cross-Site Scripting
(XSS) and Clickjacking.",
                remediation="Implement: default-src 'self'; script-src 'self'; object-src 'none';"
            ),
            HeaderFinding(
                header="Strict-Transport-Security",
                status="FAIL",
                severity=Severity.HIGH,
                value=None,
                description="Missing HSTS header on HTTPS connection. Vulnerable to SSL-stripping downg
rade attacks.",
                remediation="Add: Strict-Transport-Security: max-age=31536000; includeSubDomains; prelo
ad"
            ),
            HeaderFinding(
                header="X-Frame-Options",
                status="FAIL",
                severity=Severity.MEDIUM,
                value=None,
                description="Missing X-Frame-Options. Page can be framed by arbitrary sites for Clickja
cking attacks.",
                remediation="Set: X-Frame-Options: DENY"
            ),
            HeaderFinding(
                header="X-Content-Type-Options",
                status="PASS",
                severity=Severity.PASS,
                value="nosniff",
                description="MIME-sniffing protection is properly enabled ('nosniff').",
                remediation="Maintain existing header configuration."
            ),
            HeaderFinding(
                header="Information Leakage",
                status="FAIL",
                severity=Severity.LOW,
                value="server: Apache/2.4.41 (Ubuntu); x-powered-by: PHP/7.4.3",
                description="Server exposes internal software versions in response headers.",
```

```
                remediation="Suppress server banners in apache2.conf and set expose_php = Off."
            )
        ],
        cors_findings=[
            CORSFinding(
                probe_type="Arbitrary Origin Reflection with Credentials (CRITICAL)",
                test_origin="https://attacker-security-test.com",
                severity=Severity.CRITICAL,
                vulnerable=True,
                description="Server reflects arbitrary Origin 'https://attacker-security-test.com' AND
sets Access-Control-Allow-Credentials: true. Allows attackers to steal sensitive user data via cross-si
te fetch.",
                acao_header="https://attacker-security-test.com",
                acac_header="true",
                remediation="Implement strict server-side whitelist matching against trusted domains. N
ever dynamically reflect untrusted origins with credentials."
            ),
            CORSFinding(
                probe_type="Null Origin Trust",
                test_origin="null",
                severity=Severity.HIGH,
                vulnerable=True,
                description="Server explicitly trusts 'null' origin. Sandboxed iframe attacks can extra
ct protected data.",
                acao_header="null",
                acac_header="false",
                remediation="Never whitelist or return 'Access-Control-Allow-Origin: null'."
            )
        ]
    )

    res2 = ScanResult(
        target_url="https://secure-bank-app.local",
        scan_timestamp=datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        status_code=200,
        response_time_ms=88.2,
        security_score=98,
        grade="A+",
        header_findings=[
            HeaderFinding(
                header="Content-Security-Policy",
                status="PASS",
                severity=Severity.PASS,
                value="default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'non
e';",
                description="Strong Content-Security-Policy header is configured.",
                remediation="Maintain existing restrictive CSP directives."
            ),
            HeaderFinding(
                header="Strict-Transport-Security",
                status="PASS",
                severity=Severity.PASS,
                value="max-age=31536000; includeSubDomains; preload",
                description="Strong HSTS configuration enforces encrypted connections.",
                remediation="Keep HSTS enabled and maintain preload eligibility."
            ),
            HeaderFinding(
                header="X-Frame-Options",
                status="PASS",
                severity=Severity.PASS,
                value="DENY",
                description="Properly configured with 'DENY' to prevent UI redressing / clickjacking.",
                remediation="Keep existing policy in place."
            ),
            HeaderFinding(
                header="X-Content-Type-Options",
                status="PASS",
```

```
                severity=Severity.PASS,
                value="nosniff",
                description="MIME-sniffing protection is properly enabled ('nosniff').",
                remediation="Maintain existing header configuration."
            ),
            HeaderFinding(
                header="Referrer-Policy",
                status="PASS",
                severity=Severity.PASS,
                value="strict-origin-when-cross-origin",
                description="Secure Referrer-Policy restricts referral data leakage.",
                remediation="Maintain safe Referrer-Policy."
            ),
            HeaderFinding(
                header="Information Leakage",
                status="PASS",
                severity=Severity.PASS,
                value="None detected",
                description="No sensitive server banners or software version headers detected.",
                remediation="Maintain banner suppression."
            )
        ],
        cors_findings=[
            CORSFinding(
                probe_type="CORS Security Baseline",
                test_origin="N/A",
                severity=Severity.PASS,
                vulnerable=False,
                description="Endpoint does not insecurely reflect arbitrary origins or trust 'null' ori
gin.",
                acao_header=None,
                acac_header=None,
                remediation="Maintain strict origin whitelisting on all sensitive API endpoints."
            )
        ]
    )
    return [res1, res2]

# ==============================================================================
# Reporting Engines: Console, JSON, HTML
# ==============================================================================

class Reporter:
    """Renders scan results in Console (ANSI), JSON, or HTML formats."""

    @staticmethod
    def render_console(results: List[ScanResult], use_color: bool = True):
        banner = """
======================================================================
     AUTOMATED WEB SCANNER: SECURE HEADERS & CORS FLAWS AUDITOR
                  IBM SkillsBuild Cybersecurity Bootcamp
======================================================================
"""
        print(colorize(banner, Colors.CYAN + Colors.BOLD, use_color))

        for idx, res in enumerate(results, 1):
            grade_color = Colors.GREEN if res.grade in ["A+", "A"] else (Colors.YELLOW if res.grade in
["B", "C"] else Colors.RED)
            print("-" * 70)
            print(f"[{idx}/{len(results)}] Target: {colorize(res.target_url, Colors.BOLD + Colors.WHIT
E, use_color)}")
            print(f"Scan Time:     {res.scan_timestamp} | HTTP Status: {res.status_code} | Latency: {re
s.response_time_ms} ms")
            print(f"Security Score: {colorize(f'{res.security_score}/100 (Grade: {res.grade})', grade_c
olor + Colors.BOLD, use_color)}")
            print("-" * 70)
```

```
            if res.errors:
                for err in res.errors:
                    print(colorize(f"  [!] Error: {err}", Colors.RED, use_color))
                continue

            print(colorize("\n--- [1] HTTP SECURE HEADERS AUDIT ---", Colors.BOLD + Colors.CYAN, use_co
lor))
            for f in res.header_findings:
                if f.status == "PASS":
                    tag = colorize("[PASS]", Colors.GREEN + Colors.BOLD, use_color)
                elif f.status == "WARN":
                    tag = colorize("[WARN]", Colors.YELLOW + Colors.BOLD, use_color)
                elif f.status == "FAIL":
                    tag = colorize("[FAIL]", Colors.RED + Colors.BOLD, use_color)
                else:
                    tag = colorize("[INFO]", Colors.BLUE + Colors.BOLD, use_color)

                print(f"  {tag} {colorize(f.header, Colors.BOLD, use_color)} (Severity: {f.severity})")
                print(f"         {f.description}")
                if f.remediation and f.status != "PASS":
                    print(colorize(f"         Fix: {f.remediation}", Colors.DIM + Colors.CYAN, use_colo
r))

            print(colorize("\n--- [2] CORS MISCONFIGURATION PROBING ---", Colors.BOLD + Colors.CYAN, us
e_color))
            for c in res.cors_findings:
                if c.vulnerable:
                    tag = colorize(f"[{c.severity}]", Colors.RED + Colors.BOLD, use_color)
                else:
                    tag = colorize("[SAFE]", Colors.GREEN + Colors.BOLD, use_color)

                print(f"  {tag} {colorize(c.probe_type, Colors.BOLD, use_color)}")
                print(f"         Test Origin: {c.test_origin}")
                print(f"         Finding:     {c.description}")
                if c.acao_header:
                    print(f"         ACAO Header: {c.acao_header} | ACAC Header: {c.acac_header}")
                if c.vulnerable:
                    print(colorize(f"         Remediation: {c.remediation}", Colors.YELLOW, use_color))
            print()

        print("=" * 70)
        print(f"Audit completed. Processed {len(results)} target(s).")
        print("=" * 70)

    @staticmethod
    def render_json(results: List[ScanResult]) -> str:
        data = {
            "scanner_version": "1.0.0",
            "project": "IBM SkillsBuild Cybersecurity Bootcamp",
            "report_generated_at": datetime.datetime.now().isoformat(),
            "target_count": len(results),
            "results": [asdict(r) for r in results]
        }
        return json.dumps(data, indent=2)

    @staticmethod
    def render_html(results: List[ScanResult]) -> str:
        results_html = ""
        total_vulns = sum(
            sum(1 for f in r.header_findings if f.status in ["FAIL", "WARN"]) +
            sum(1 for c in r.cors_findings if c.vulnerable)
            for r in results
        )

        for res in results:
            grade_class = "grade-a" if res.grade in ["A+", "A"] else ("grade-b" if res.grade in ["B",
"C"] else "grade-f")
```

```
            header_rows = ""
            for h in res.header_findings:
                badge_class = f"badge-{h.status.lower()}"
                header_rows += f"""
                <tr>
                    <td><strong>{html.escape(h.header)}</strong></td>
                    <td><span class="badge {badge_class}">{html.escape(h.status)}</span></td>
                    <td><span class="badge badge-sev-{h.severity.lower()}">{html.escape(h.severity)}</s
pan></td>
                    <td>{html.escape(h.description)}</td>
                    <td class="code-cell">{html.escape(h.remediation)}</td>
                </tr>
                """

            cors_rows = ""
            for c in res.cors_findings:
                status_text = "VULNERABLE" if c.vulnerable else "SECURE"
                badge_class = "badge-fail" if c.vulnerable else "badge-pass"
                acao_val = html.escape(c.acao_header or "None")
                acac_val = html.escape(c.acac_header or "N/A")
                cors_rows += f"""
                <tr>
                    <td><strong>{html.escape(c.probe_type)}</strong></td>
                    <td><code>{html.escape(c.test_origin)}</code></td>
                    <td><span class="badge {badge_class}">{status_text}</span></td>
                    <td><span class="badge badge-sev-{c.severity.lower()}">{html.escape(c.severity)}</s
pan></td>
                    <td>{html.escape(c.description)}<br><small><strong>ACAO:</strong> {acao_val} | <str
ong>ACAC:</strong> {acac_val}</small></td>
                    <td class="code-cell">{html.escape(c.remediation)}</td>
                </tr>
                """

            results_html += f"""
            <div class="card">
                <div class="card-header">
                    <div>
                        <h2 class="target-title">{html.escape(res.target_url)}</h2>
                        <div class="meta-line">Scan Time: {res.scan_timestamp} | HTTP Status: {res.stat
us_code} | Latency: {res.response_time_ms} ms</div>
                    </div>
                    <div class="score-box {grade_class}">
                        <div class="score-value">{res.security_score}/100</div>
                        <div class="score-label">Grade: {res.grade}</div>
                    </div>
                </div>

                <h3>HTTP Security Headers Audit</h3>
                <div class="table-responsive">
                    <table>
                        <thead>
                            <tr>
                                <th>Header</th>
                                <th>Status</th>
                                <th>Severity</th>
                                <th>Finding & Impact</th>
                                <th>Remediation Guide</th>
                            </tr>
                        </thead>
                        <tbody>
                            {header_rows}
                        </tbody>
                    </table>
                </div>

                <h3>Cross-Origin Resource Sharing (CORS) Audit</h3>
```

```
                <div class="table-responsive">
                    <table>
                        <thead>
                            <tr>
                                <th>Probe Scenario</th>
                                <th>Test Origin</th>
                                <th>Outcome</th>
                                <th>Severity</th>
                                <th>Observation</th>
                                <th>Remediation Guide</th>
                            </tr>
                        </thead>
                        <tbody>
                            {cors_rows}
                        </tbody>
                    </table>
                </div>
            </div>
            """

        html_template = f"""<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Web Security Audit Report - IBM SkillsBuild Cybersecurity</title>
    <style>
        :root {{
            --bg: #f8fafc;
            --surface: #ffffff;
            --primary: #0f62fe;
            --text-dark: #1e293b;
            --text-muted: #64748b;
            --border: #e2e8f0;
            --pass: #198038;
            --warn: #f1c21b;
            --fail: #da1e28;
            --critical: #8a0e14;
        }}
        body {{
            font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-
serif;
            background-color: var(--bg);
            color: var(--text-dark);
            margin: 0;
            padding: 30px 20px;
            line-height: 1.5;
        }}
        .container {{
            max-width: 1200px;
            margin: 0 auto;
        }}
        header {{
            background: linear-gradient(135deg, #121619 0%, #0043ce 100%);
            color: white;
            padding: 30px;
            border-radius: 12px;
            margin-bottom: 25px;
            box-shadow: 0 4px 12px rgba(0,0,0,0.1);
        }}
        header h1 {{ margin: 0 0 10px 0; font-size: 26px; }}
        header p {{ margin: 0; opacity: 0.9; font-size: 15px; }}
        .summary-stats {{
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
            gap: 15px;
            margin-bottom: 25px;
        }}
```

```
            .stat-card {{
                background: var(--surface);
                padding: 20px;
                border-radius: 8px;
                border: 1px solid var(--border);
                text-align: center;
            }}
            .stat-card .num {{ font-size: 28px; font-weight: bold; color: var(--primary); }}
            .stat-card .label {{ color: var(--text-muted); font-size: 13px; text-transform: uppercase; marg
    in-top: 5px; }}
            .card {{
                background: var(--surface);
                border-radius: 10px;
                border: 1px solid var(--border);
                padding: 25px;
                margin-bottom: 30px;
                box-shadow: 0 2px 8px rgba(0,0,0,0.04);
            }}
            .card-header {{
                display: flex;
                justify-content: space-between;
                align-items: center;
                border-bottom: 1px solid var(--border);
                padding-bottom: 15px;
                margin-bottom: 20px;
            }}
            .target-title {{ margin: 0; font-size: 20px; color: var(--primary); }}
            .meta-line {{ font-size: 13px; color: var(--text-muted); margin-top: 4px; }}
            .score-box {{
                padding: 10px 20px;
                border-radius: 8px;
                text-align: center;
                color: white;
            }}
            .grade-a {{ background: #198038; }}
            .grade-b {{ background: #f1c21b; color: #1e293b; }}
            .grade-f {{ background: #da1e28; }}
            .score-value {{ font-size: 24px; font-weight: bold; }}
            .score-label {{ font-size: 12px; text-transform: uppercase; }}
            h3 {{ margin: 25px 0 12px 0; font-size: 17px; border-left: 4px solid var(--primary); padding-le
    ft: 10px; }}
            .table-responsive {{ overflow-x: auto; }}
            table {{
                width: 100%;
                border-collapse: collapse;
                font-size: 13.5px;
                margin-bottom: 15px;
            }}
            th, td {{
                padding: 10px 12px;
                border: 1px solid var(--border);
                text-align: left;
                vertical-align: top;
            }}
            th {{ background: #f1f5f9; font-weight: 600; color: #475569; }}
            .badge {{
                display: inline-block;
                padding: 3px 8px;
                border-radius: 4px;
                font-size: 11px;
                font-weight: bold;
                text-transform: uppercase;
            }}
            .badge-pass {{ background: #defbe6; color: #198038; }}
            .badge-warn {{ background: #fef3c7; color: #b45309; }}
            .badge-fail {{ background: #ffe3e3; color: #da1e28; }}
            .badge-info {{ background: #e0f2fe; color: #0369a1; }}
```

```
        .badge-sev-critical {{ background: #7f1d1d; color: white; }}
        .badge-sev-high {{ background: #dc2626; color: white; }}
        .badge-sev-medium {{ background: #f59e0b; color: white; }}
        .badge-sev-low {{ background: #64748b; color: white; }}
        .code-cell {{ font-family: monospace; font-size: 12px; color: #0f62fe; word-break: break-all;
}}
        footer {{ text-align: center; font-size: 12px; color: var(--text-muted); margin-top: 40px; }}
    </style>
</head>
<body>
    <div class="container">
        <header>
            <h1>Automated Web Security Audit Report</h1>
            <p>IBM SkillsBuild National Cybersecurity Bootcamp | Project: Secure Headers & CORS Audit S
canner</p>
        </header>

        <div class="summary-stats">
            <div class="stat-card">
                <div class="num">{len(results)}</div>
                <div class="label">Targets Audited</div>
            </div>
            <div class="stat-card">
                <div class="num">{total_vulns}</div>
                <div class="label">Total Findings</div>
            </div>
            <div class="stat-card">
                <div class="num">{datetime.datetime.now().strftime("%Y-%m-%d")}</div>
                <div class="label">Audit Date</div>
            </div>
        </div>

        {results_html}

        <footer>
            Generated by Automated Web Security Scanner &bull; IBM SkillsBuild Bootcamp Project Deliver
able
        </footer>
    </div>
</body>
</html>
"""
        return html_template

# ==============================================================================
# CLI Entry Point
# ==============================================================================

def main():
    parser = argparse.ArgumentParser(
        description="Automated Web Scanner: Audit sites for secure headers and CORS flaws (IBM SkillsBu
ild Cybersecurity Project)"
    )
    parser.add_argument("-u", "--url", type=str, help="Single target web application URL (e.g. https://
example.com)")
    parser.add_argument("-f", "--file", type=str, help="Path to text file containing target URLs (one p
er line)")
    parser.add_argument("-o", "--output", type=str, help="Output file path to save report (.json or .ht
ml)")
    parser.add_argument("--format", choices=["console", "json", "html"], default="console", help="Repor
t format (default: console)")
    parser.add_argument("-t", "--threads", type=int, default=5, help="Concurrent threads for bulk scann
ing (default: 5)")
    parser.add_argument("--timeout", type=float, default=8.0, help="HTTP request timeout in seconds (de
fault: 8.0)")
    parser.add_argument("-k", "--insecure", action="store_true", help="Disable SSL certificate verifica
tion (testing only)")
```

```
    parser.add_argument("--no-color", action="store_true", help="Disable ANSI colored terminal output")
    parser.add_argument("--demo", action="store_true", help="Run in offline demonstration mode with moc
k vulnerable & secure targets")

    args = parser.parse_args()

    # 1. Handle Demo Mode
    if args.demo:
        print("[*] Running in Offline Demonstration Mode...")
        results = generate_demo_results()
    else:
        urls: List[str] = []
        if args.url:
            urls.append(args.url.strip())
        elif args.file:
            if not os.path.exists(args.file):
                print(f"[!] Error: File not found: {args.file}", file=sys.stderr)
                sys.exit(1)
            with open(args.file, "r", encoding="utf-8") as f:
                urls = [line.strip() for line in f if line.strip() and not line.startswith("#")]
        else:
            parser.print_help()
            print("\n[!] Tip: Provide either --url <URL>, --file <FILE>, or test with --demo", file=sy
s.stderr)
            sys.exit(1)

        scanner = WebScanner(timeout=args.timeout, insecure_ssl=args.insecure, max_threads=args.thread
s)
        if len(urls) == 1:
            results = [scanner.scan_url(urls[0])]
        else:
            results = scanner.scan_bulk(urls)

    # 2. Output Handling
    enable_color = not args.no_color
    if args.format == "console" and not args.output:
        Reporter.render_console(results, use_color=enable_color)
    elif args.format == "json" or (args.output and args.output.endswith(".json")):
        json_content = Reporter.render_json(results)
        if args.output:
            with open(args.output, "w", encoding="utf-8") as f:
                f.write(json_content)
            print(f"[+] Successfully exported JSON audit report to: {args.output}")
        else:
            print(json_content)
    elif args.format == "html" or (args.output and args.output.endswith(".html")):
        html_content = Reporter.render_html(results)
        if args.output:
            with open(args.output, "w", encoding="utf-8") as f:
                f.write(html_content)
            print(f"[+] Successfully exported HTML audit report to: {args.output}")
        else:
            print(html_content)
    else:
        Reporter.render_console(results, use_color=enable_color)
        if args.output:
            with open(args.output, "w", encoding="utf-8") as f:
                f.write(Reporter.render_json(results))
            print(f"[+] Saved scan data to {args.output}")

if __name__ == "__main__":
    main()
```